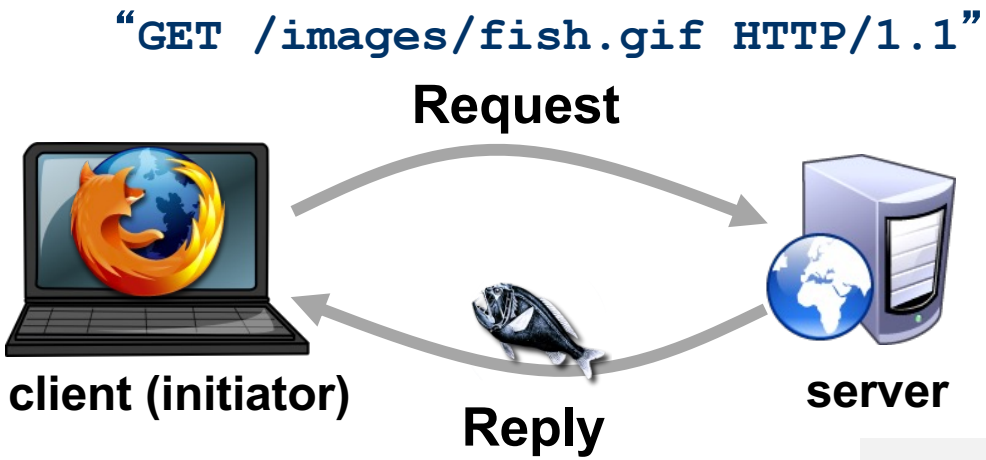
CompSci 390-03-26S

# Distributed and Networked Systems

# RPC and Server Performance

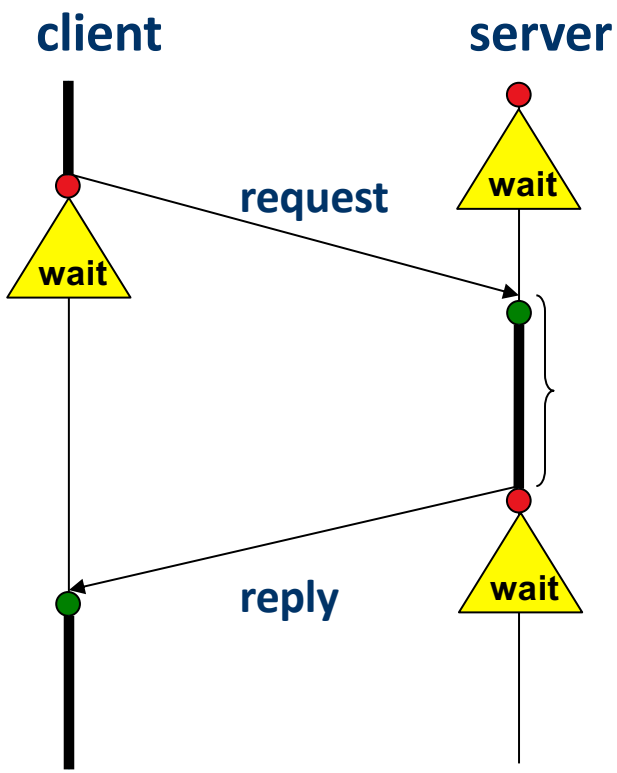
# Clients, servers, and the socket API

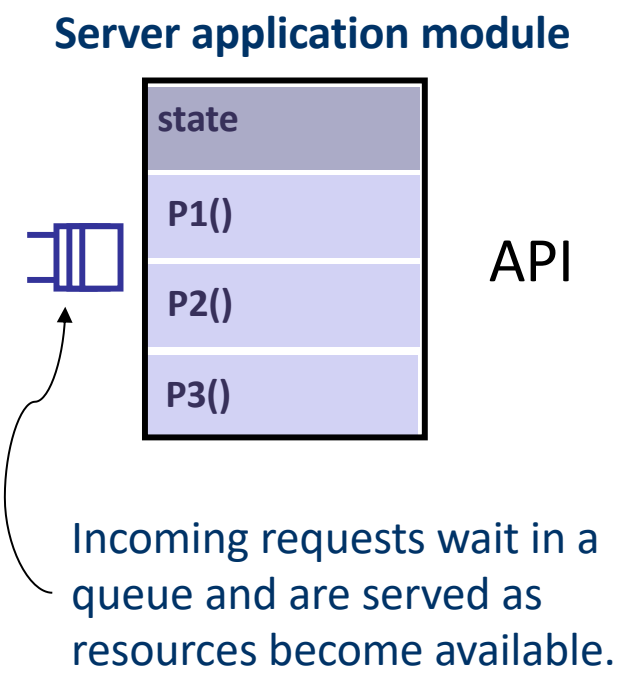



```
sd = socket(…);
connect(sd, name+port);
write(sd, request…);
read(sd, reply…);
close(sd);
```

```
s = socket(…);
bind(s, name+port);
listen(s, N);
sd = accept(s);
read(sd, request…);
write(sd, reply…);
close(sd);
```

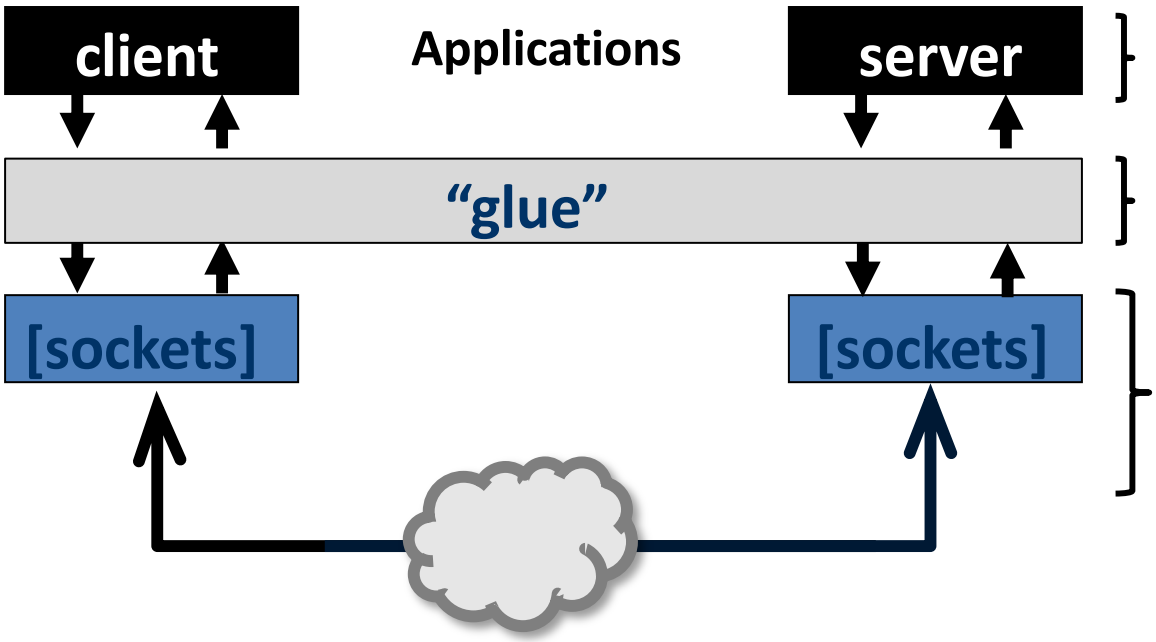
# Client/server abstraction



# Remote Procedure Call (RPC)

- "RPC is a canonical structuring paradigm for client/server services."
- First saw wide use in 1980s client/server (e.g., Network File System)




Humans focus on getting this code right.

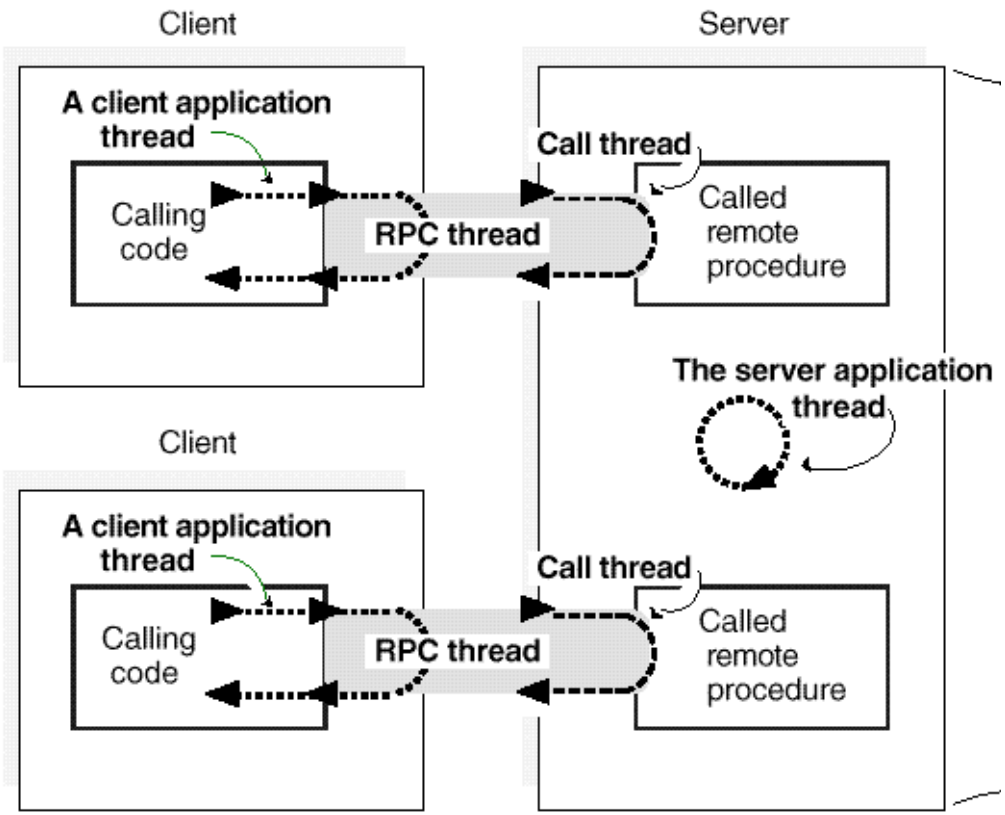
Auto-generate **stubs** from API spec (in IDL: Interface Description Language).

This code is "canned".

# RPC: Original Vision

- Integrate with programming languages
- RPC is "just like" calling a module
- In fact, that's what you **are** doing
- Caller "waits" until procedure "returns"
- Pass typed arguments
- Receive typed results (return value)
- Access args/parameters/results as ordinary typed data in PL
- **Transparency**: "Hide the network from the programmer"
- What is different?

# Concurrency: threads and RPC




[OpenGroup, late 1980s]

**Server**. When an RPC call arrives, wake up a thread from the pool to handle it.

**Client**. Thread blocks until the server thread returns a response.

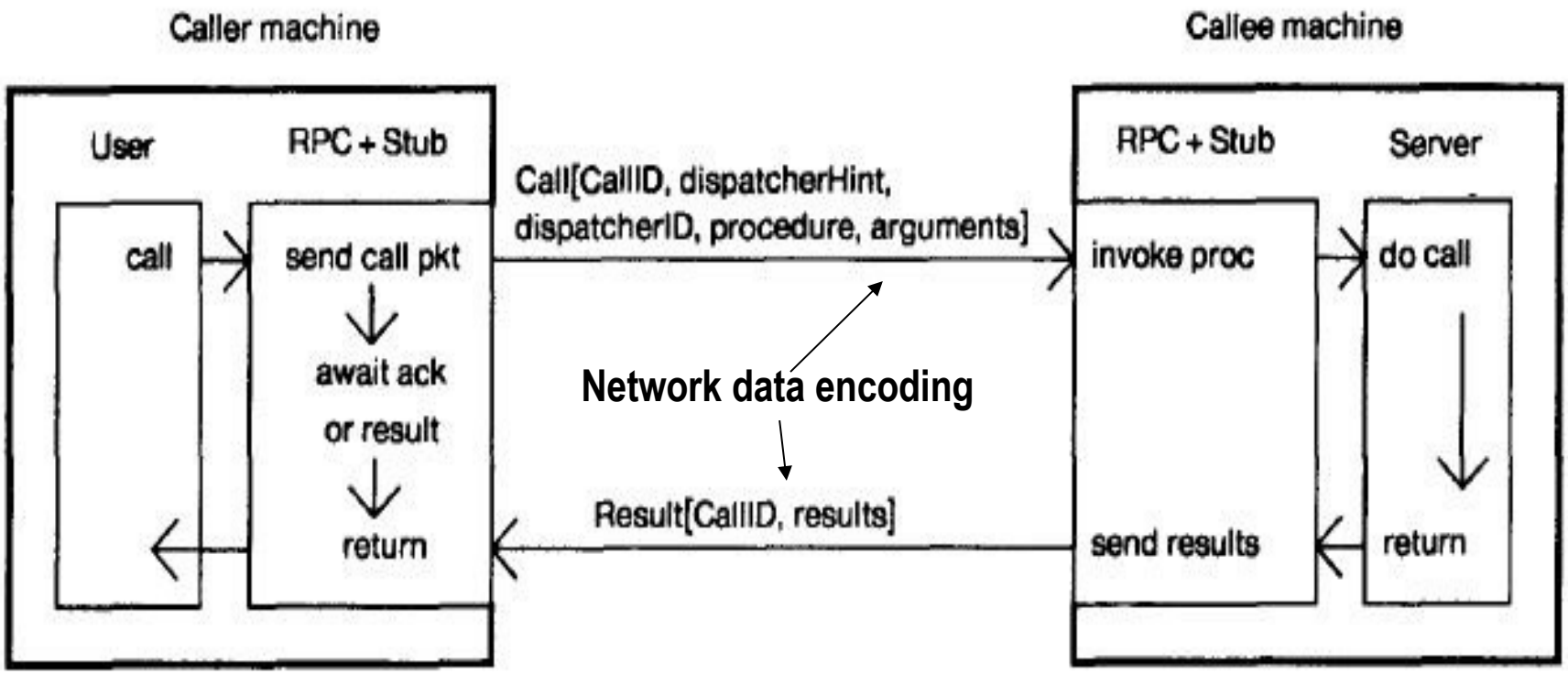
# Implementing Remote Procedure Calls

ANDREW D. BIRRELL and BRUCE JAY NELSON
Xerox Palo Alto Research Center

Remote procedure calls (RPC) appear to be a useful paradigm for providing communication across a network between programs written in a high-level language. This paper describes a package providing a remote procedure call facility, the options that face the designer of such a package, and the decisions we made. We describe the overall structure of our RPC mechanism, our facilities for binding RPC clients, the transport level communication protocol, and some performance measurements. We include descriptions of some optimizations used to achieve high performance and to minimize the load on server machines that have many clients.

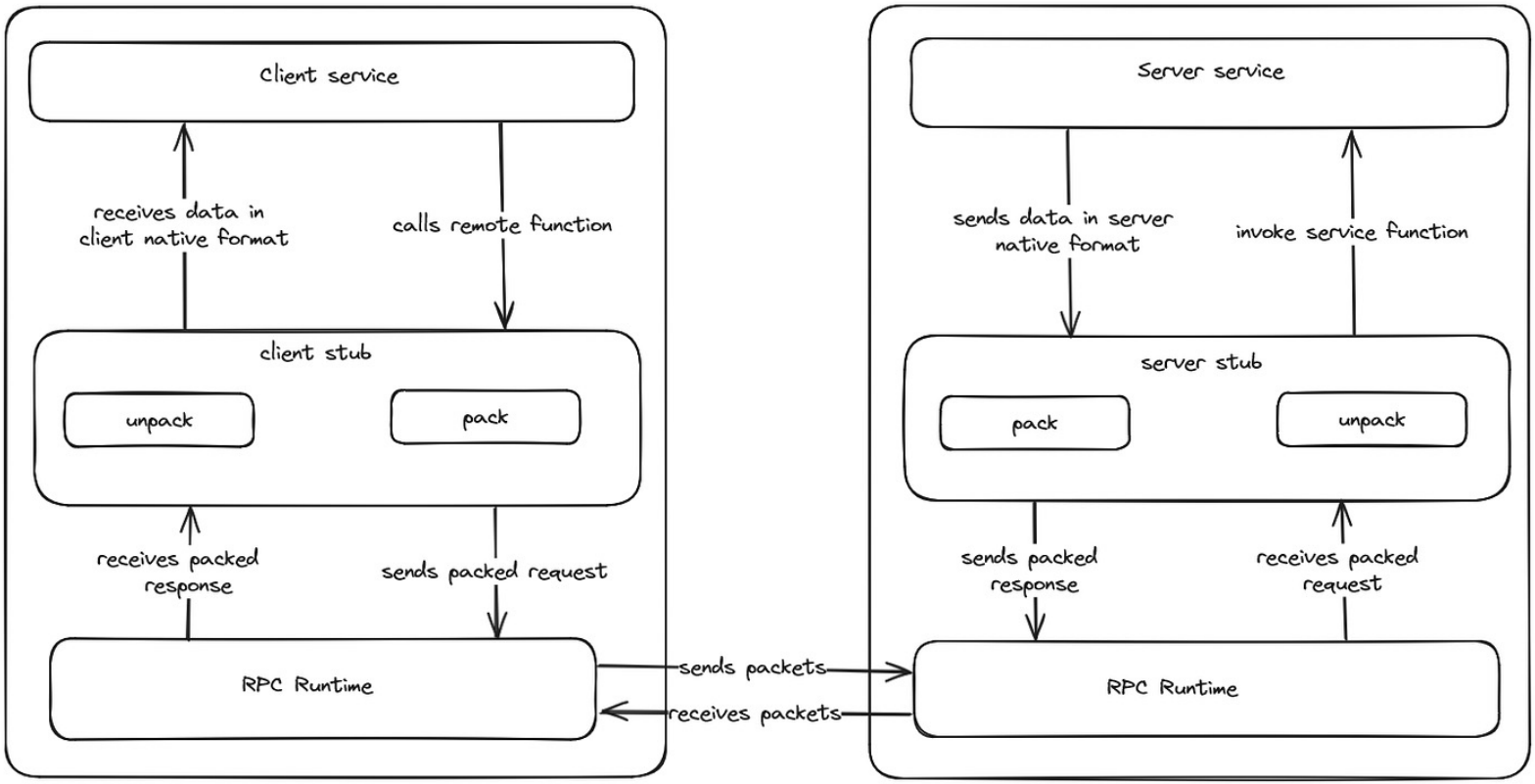
**1984. ACM SIGOPS Hall of Fame paper**
**3400 citations**

# Simple RPC Diagram




**Birrell-Nelson 1984**

# Another variant of the Big Picture




https://jadhavsaurabh037.medium.com/grpc-deep-dive-introduction-to-grpc-5aa37243c132

# The Sun Network Filesystem: Design, Implementation and Experience

*Russel Sandberg*

Sun Microsystems, Inc.
2550 Garcia Ave.
Mountain View, CA. 94043
(415) 960–7293

## Introduction

The Sun Network Filesystem (NFS™) provides transparent, remote access to filesystems. Unlike many other remote filesystem implementations under UNIX®, NFS is designed to be easily portable to other operating systems and machine architectures. It uses an External Data Representation (XDR) specification to describe protocols in a machine and system independent way. NFS is implemented on top of a Remote Procedure Call package (RPC) to help simplify protocol definition, implementation, and maintenance.

In order to build NFS into the UNIX kernel in a way that is transparent to applications, we decided to add a new interface to the kernel which separates generic filesystem operations from specific filesystem implementations. The "filesystem interface" consists of two parts: the Virtual File System (VFS)

**1985: 2000+ citations**

# But that was in the 1980s!

gRPC over HTTP/2 represents current state of the art.

- Used at Google, open-sourced by Google
- IDL and wire format for typed data elements
  - Needed for heterogeneous architectures and languages
  - Tired: Sun IDL and External Data Representation (XDR)
  - Wired: **protocol buffers** (protobuf: faster, with collection types)
- Web-capable reliable transport: HTTP over TCP

gRPC

grpc.io

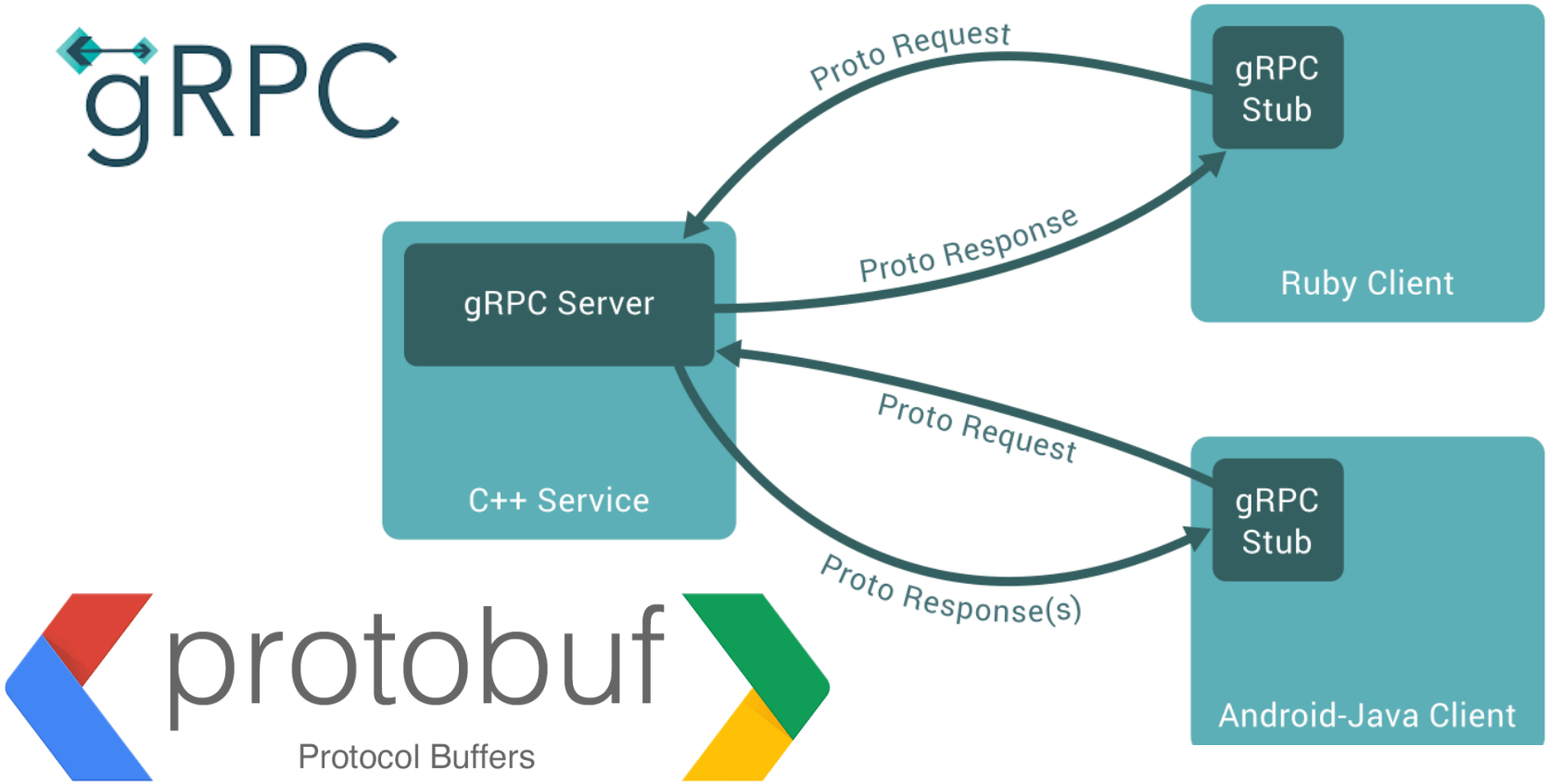
gRPC can help make connecting, operating and debugging distributed systems as easy as making local function calls; the framework handles all the complexities normally associated with enforcing strict service contracts, data serialization, efficient network communication, authentications and access control, distributed tracing and so on. gRPC along with **protocol buffers** enables loose coupling, engineering velocity, higher reliability and ease of operations. Also, gRPC allows developers to write service definitions in a language-agnostic spec and generate clients and servers in multiple languages.

Google Cloud

https://cloud.google.com/blog/products/gcp/grpc-a-true-internet-scale-rpc-framework-is-now-1-and-ready-for-production-deployments

**2020**

# Language independence



# Server API/body

```
func (s *greeterServer) Greet(
    ctx context.Context,
    req *greetpb.GreetRequest,
) (*greetpb.GreetResponse, error) {
    return &greetpb.GreetResponse{
        Message: "Hello " + req.Name + "!",
    }, nil
}
```

# Server main

```
func main() {
    lis, err := net.Listen("tcp", "localhost:50051")
    ...
    grpcServer := grpc.NewServer()
    ...RegisterGreeterServer(....)

    grpcServer.Serve(lis)
    ...
}
```

# Client main

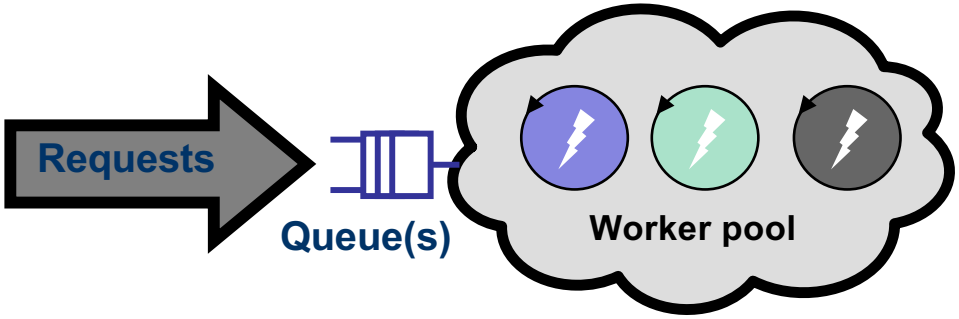
```
func main() {
    conn := grpc.Dial("localhost:50051",...)
    ...
    defer conn.Close()

    client := greetpb.NewGreeterClient(conn)

    ctx, cancel := context.WithTimeout(...)
    defer cancel()

    req := &greetpb.GreetRequest{ Name: ”Chase", }
    resp, err := client.Greet(ctx, req)
   ...
}
```

# Server performance: Getting work done

- Consider a stream of requests to a server.
- Performance **metrics:**
  - **Service demand**. How long does it occupy server?
  - **Response time.** How long from request to reply?
    - Service demand + waiting time (queue delay)
    - Queues everywhere!
  - **Throughput**. How many complete per unit of time?
  - **Peak** throughput: throughput at capacity.
    - **Saturation**: server capacity is fully busy.

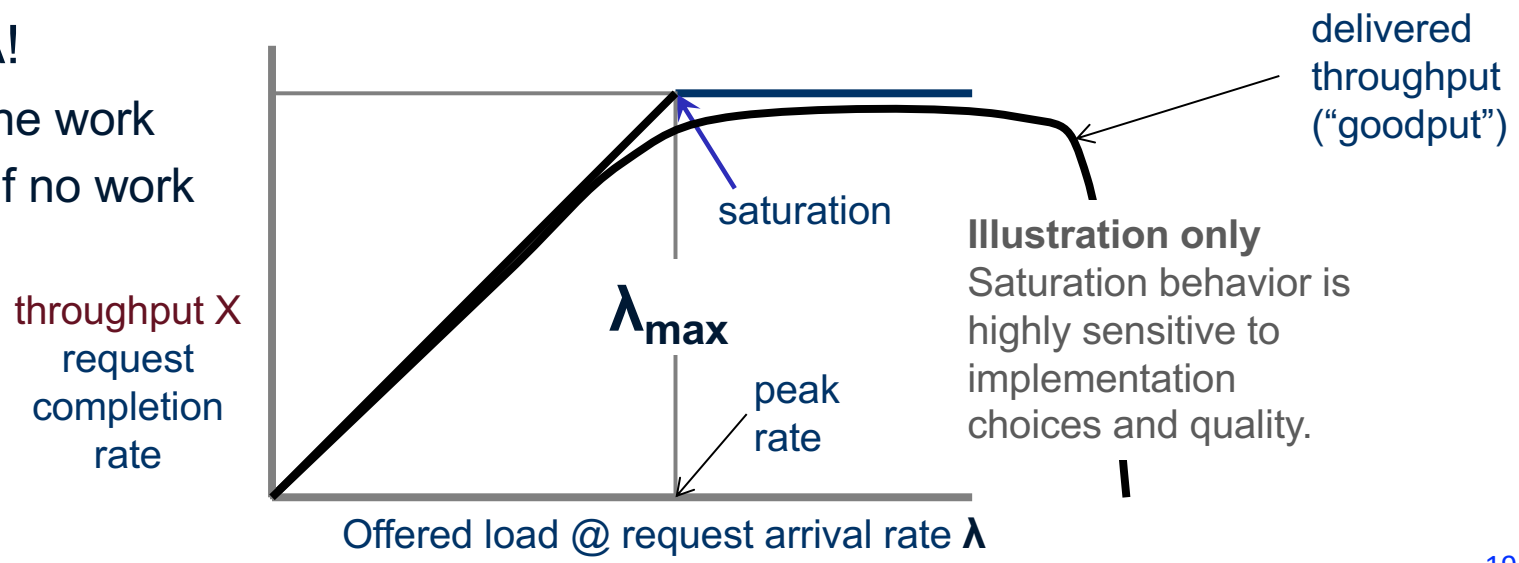
# Labs 2-3 (servers, flows) illustrate queue behavior

- Lab #2 (Servers) illustrates queue behavior in an empirical real system whose behavior (mostly) matches the theory.
  - $R = D/(1-U)$ where $U = \lambda D$
- But mean R grows more slowly with more server concurrency.
  - c threads/CPUs → M/M/c queue model (details out of scope)



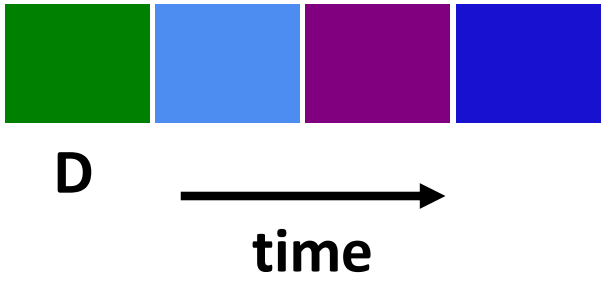
# Offered load and throughput

- **Objective**: predict server performance under any **offered load**.
  - Given **request mix**: ops, costs, sizes, etc. — depends on app service
  - A request **arrival rate**, called lambda ($\lambda$), e.g., in requests/second.
- What is throughput X when $\lambda$ is **below** the peak rate $\lambda_{max}$?
  - X = $\lambda$!
  - Do the work
  - Idle if no work



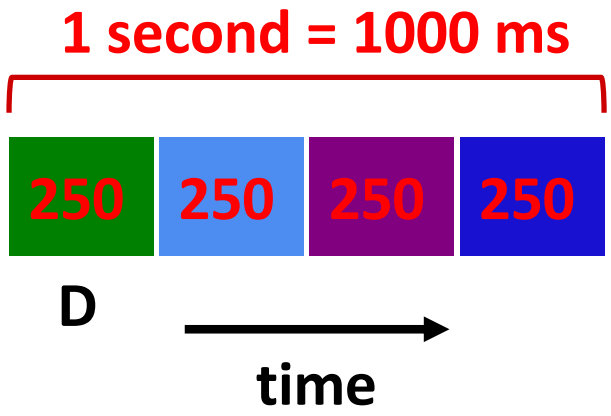
# About time: intervals and rates

- Suppose requests execute **serially**: one at a time.
- Suppose each request executes for service time D (demand).
- What is the **peak throughput** X of this system?
- **Example**: D = 250 milliseconds—1/4 second



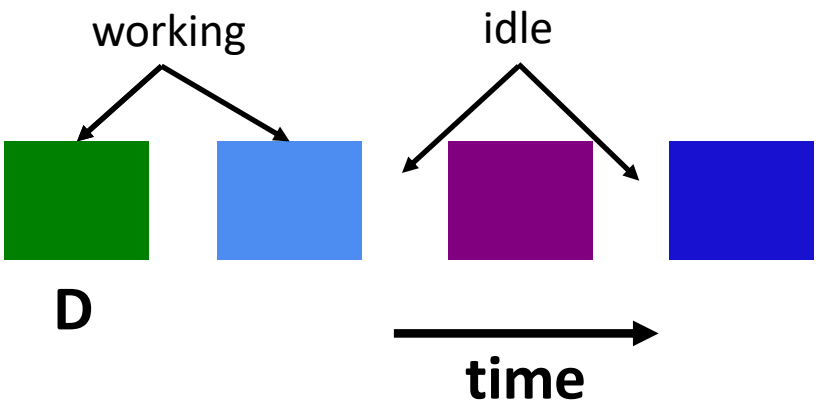
# About time: intervals and rates

- Suppose requests execute **serially**: one at a time.
- Suppose each request executes for service time D (demand).
- What is the **peak throughput** X of this system? (peak rate)
- **Example**: D = 250 milliseconds—1/4 second
- X = 4 requests/second
- It's the **inverse**! **X = 1/D**
- (When the units match.)

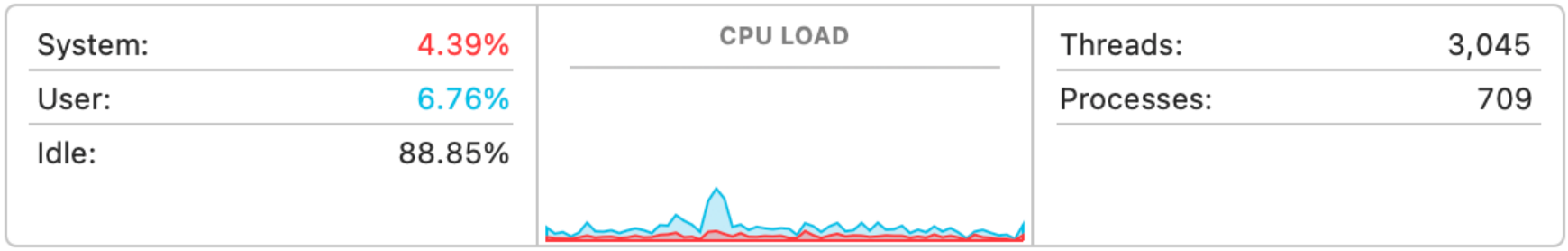


# Utilization

- **Utilization U**: % of time working (a number between 0 and 1)
- Also equivalent to **load factor**: $\lambda/\lambda_{max}$
- Back of napkin: U= XD = $\lambda$D
- What if $\lambda > \lambda_{max}$?
  - U=1 → resource is **saturated**.
  - Q: What happens when saturated?
  - **Answer**: requests arriving when the resource is **busy** wait in a **queue**.
  - Queues grow faster at higher U!



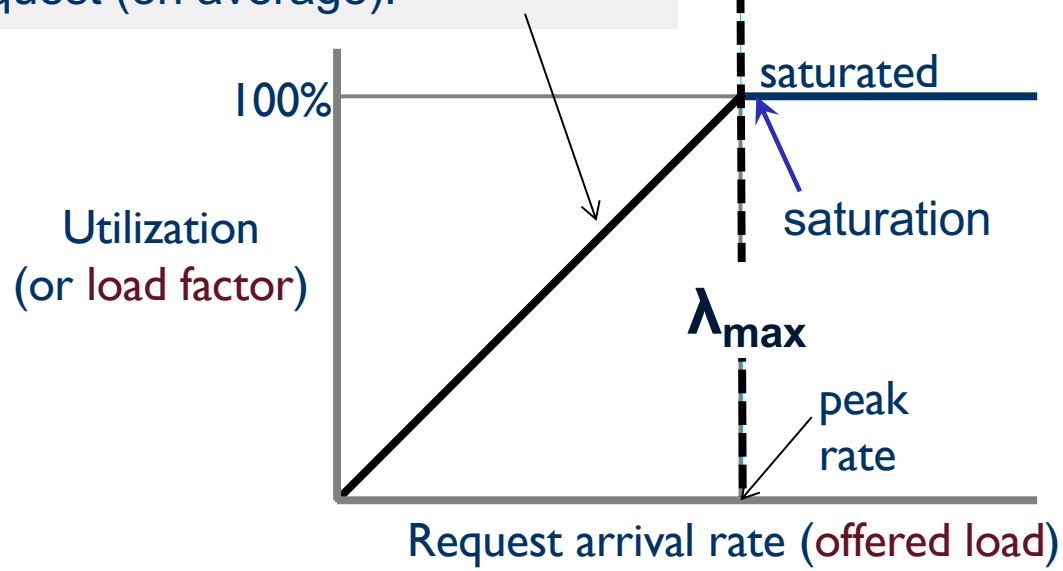
# CPU utilization on my laptop: a moment in time



# Utilization (load factor)

**U = XD**
X = throughput
D = service demand, i.e., how much time/work to complete each request (on average).

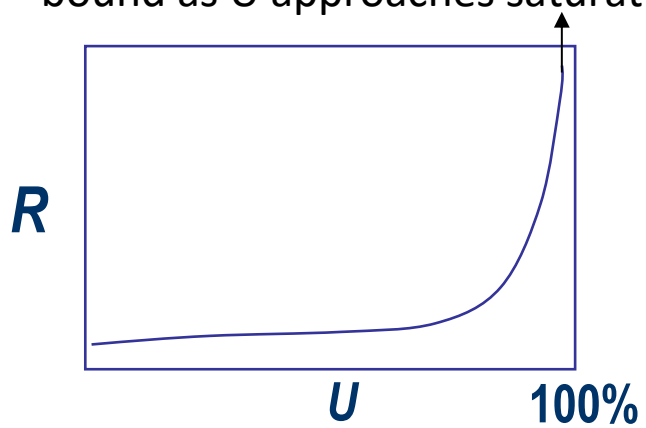
**U = 1 = 100%**
The server is saturated.
It has no spare capacity.
It is busy all the time.



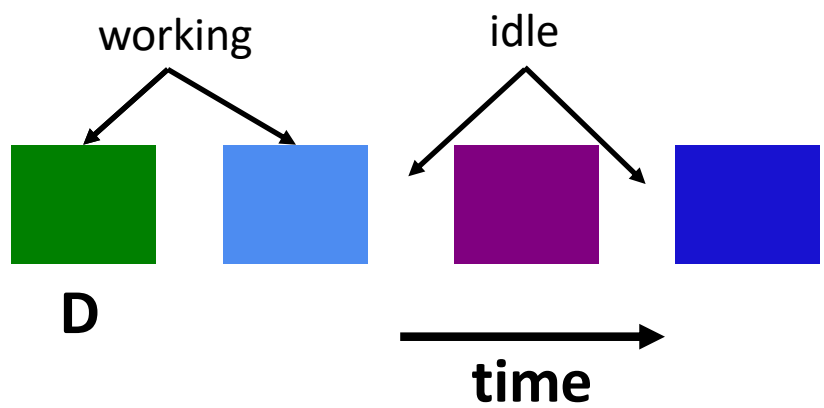

**Idealized**: serial requests with mean demand D time units on a single service center (e.g., a single CPU core).

# Queues and response time (for serial requests)

- **Idle time**: 1-U
- "Busyness" increases queue delays proportionally to 1/(1-U).
- It turns out average response time trends to R = D/(1-U).
- → Queues and delays grow without bound as U approaches saturation.




*R = D(N+1)*
Queue length N grows with U. If queue is FIFO, we need DN time on average to clear the queue, + time D for this new request.



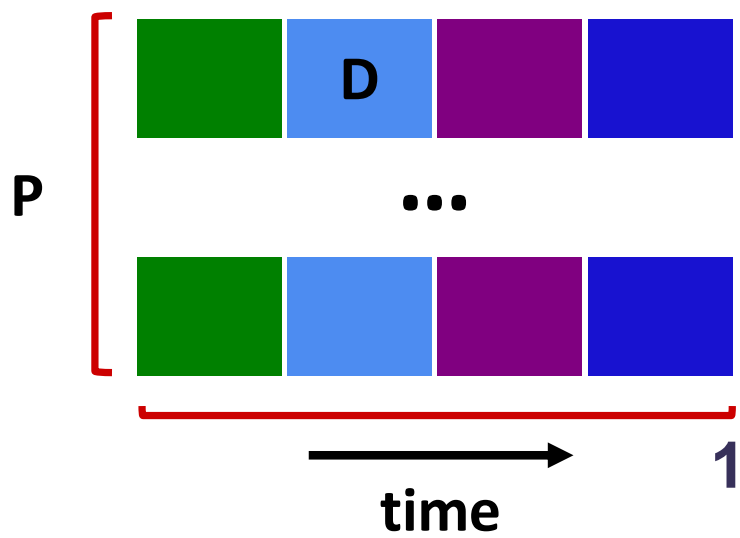
# Parallelism

- Assume: requests with mean service demand D.
- P requests can run in parallel, e.g., on P cores.
- What is the effect of parallelism on the peak throughput X?

**X = 1/D**

On one core or device

X=? on P cores/devices?

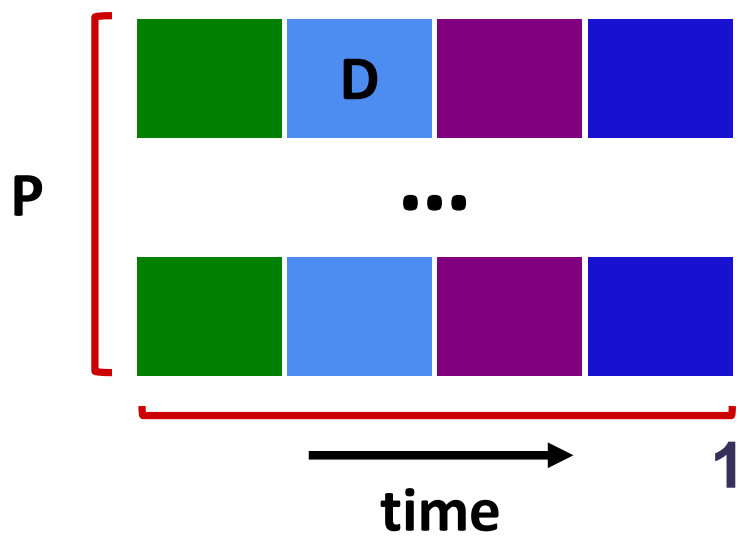


# Parallelism

- Assume: requests with mean service demand D.
- P requests can run in parallel, e.g., on P cores.
- What is the effect of parallelism on the peak throughput X?

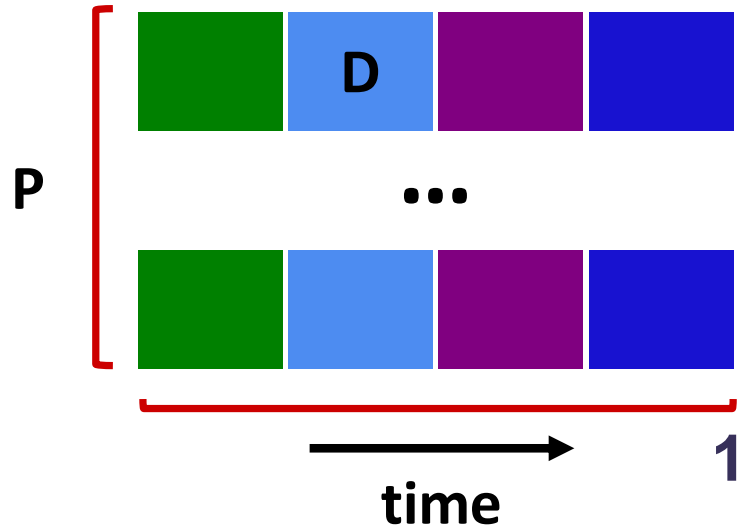
**X = P/D**

Increase by factor of P



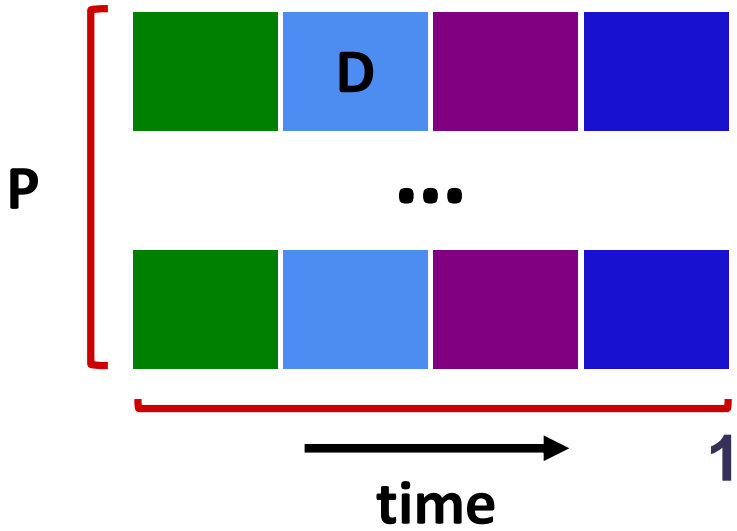
# Parallelism and response time

- Assume: requests with mean service demand D.
- P requests can execute in parallel—**serial** means P=1.
- What is the effect of P on latency or **response time R**?
  - R = from submit time to completion
  - R > D!
  - But requests could wait in a queue
  - R = D + queue delay



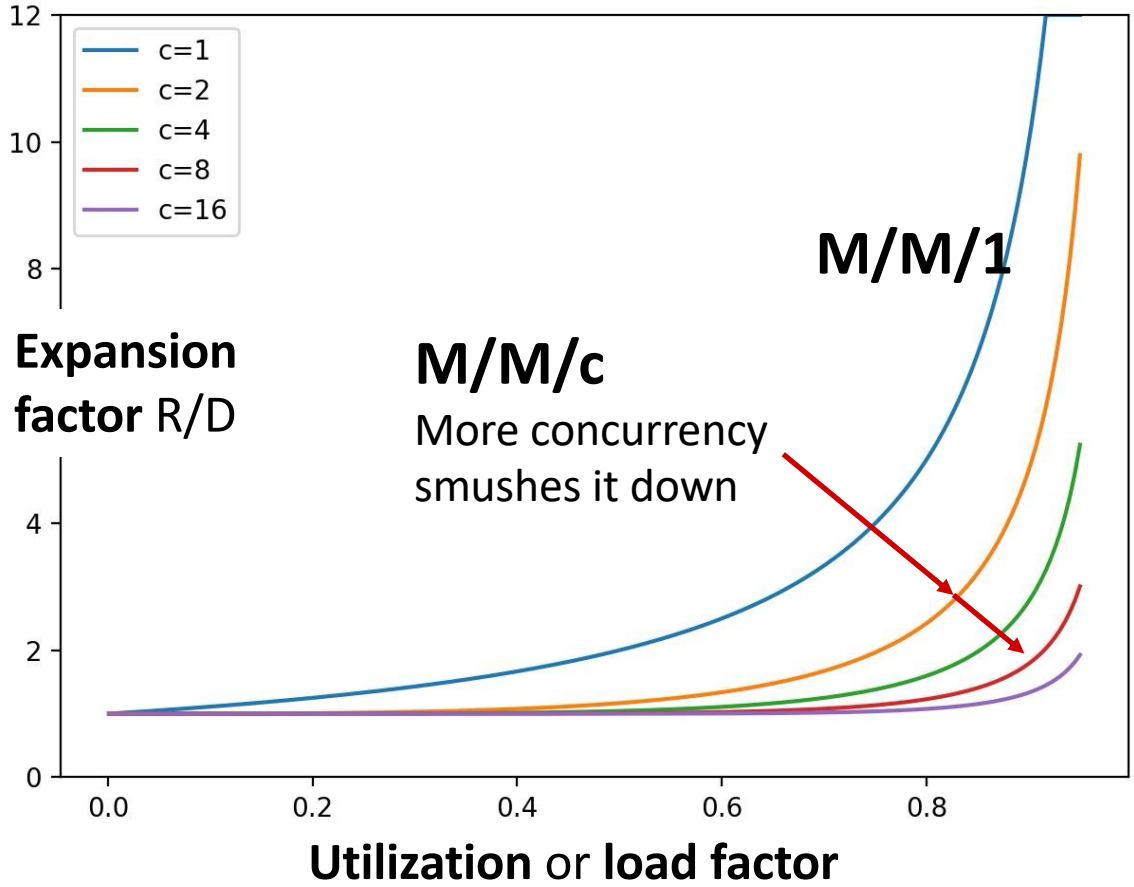
# Parallelism and response time

- Assume: requests with mean service demand D.
- P requests can execute in parallel—**serial** means P=1.
- What is the effect of P on latency or **response time R**?
- **No effect?** Still R>D
  - Parallelism "doesn't help"
  - **But**: parallelism expands capacity
  - Which reduces queue delays

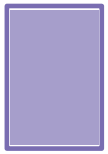


# Effect of parallelism on server queues

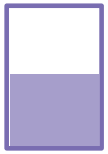
- At a given load factor, parallel servers have lower mean R.
- Other things being equal.
- But as they saturate: rapid growth and **unbounded:**
  - Mean queue length N
  - Mean response time R



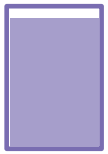
# Example: Request routing to a server set



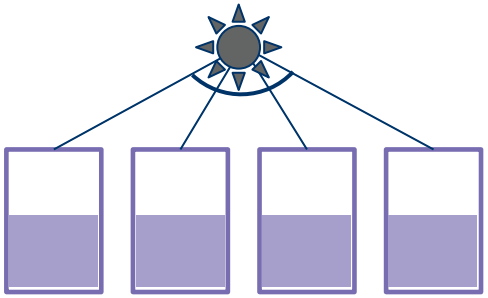
**Saturated: peak throughput λ = 100 r/s**



**λ = 50 r/s, R = 10 ms**
**U?**



**λ = 90 r/s**
**U? R?**

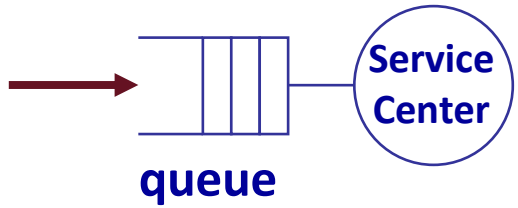


**4x**
**U=50%**
**Peak λ?**
**R?**

# Details and Derivations

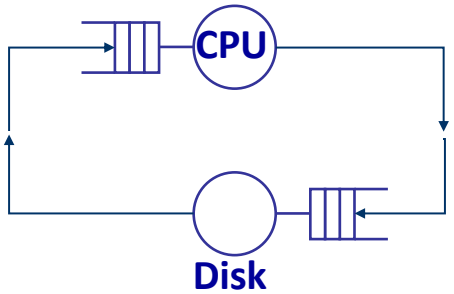
# Understanding performance: queues

**offered load**
request stream @
*arrival rate* λ



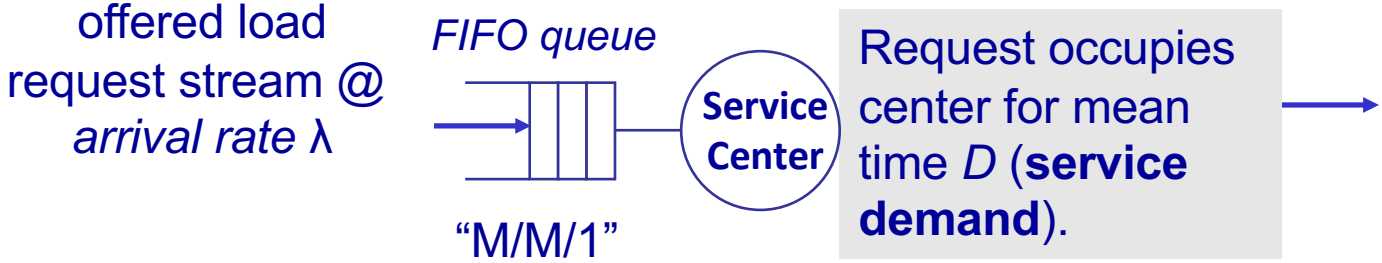

Handle request: **occupies** center for mean time *D* (service demand).

Systems are networks of centers and queues. To understand them, we must think about how centers interact.



But we can also "squint" and think of the entire server as a single queuing center, and we won't go too far astray.

# Queuing theory for busy people




- Consider: single service center (e.g., a core) with FIFO queue.
  - Independent request arrivals at mean rate λ (poisson arrivals).
  - i.e., Assume inter-arrival time (1/λ) and service demand (D) are exponentially distributed around their means. Noted as "M": M/M/1 queue.
  - For c service centers with c>1: M/M/c queue model, e.g., multithreaded server
- These assumptions are rarely exactly true for real systems, but they give a rough ("back of napkin") understanding of queue behavior.

# Little's Law

For a FIFO/FCFS queue in steady state, mean response time R and mean queue length N are governed by:

Little's Law: $N = \lambda R$

**Why?**

- Suppose a task T is in the system for R time units.
- During that time:
  - $\lambda R$ new tasks arrive (on average)
  - N tasks depart (all the tasks ahead of T, on average).
- But in steady state, the flow in balances flow out.

=

# Inverse Idle Time "Law"

- Little's Law gives mean response time R = D/(1-U)
  - "Service demand over the idle time"
- i.e., response time inflates D by **expansion factor** 1/(1-U).

Intuitively, mean response time $R = D + DN$.
**Why?** Serve the request plus N requests ahead of it in the queue, each at cost $D$.
Substituting $\lambda R$ for $N$ (by **Little's Law**): $R = D + D\lambda R$
Substituting $U$ for $\lambda D$ (by **Utilization Law**): $R = D + UR$
Do the math: $R - UR = D \rightarrow R(1 - U) = D \rightarrow R = D/(1 - U)$